

智课枢 软件说明书鉴别材料

交存方式：一般交存

文档来源：项目 README 和软件功能说明

软件名称：智课枢

软件类型：教育软件、人工智能软件

Study Agent

Study Agent 是一个面向课堂学习场景的本地优先学习系统。它把实时语音转写、视觉识别、资料解析、视频字幕、课堂问答、课后笔记和复习 cop

串成了一条完整链路。

从当前实现看，它已经不只是一个“语音转写 Demo”，而是一套围绕`course\_id + lesson\_id + session\_id`运转的课堂知识沉淀平台：

- 上课中：实时语音转写、视觉截帧 OCR/VLM、课堂内问答
- 课后：历史回看、视频字幕、精修转写、总结、测验、笔记
- 学习工作台：资料库管理、Lesson Copilot 复习编排

1. 当前能力

1.1 实时课堂

- 浏览器采集麦克风音频，通过 WebSocket 发送到后端
- 后端通过`AsrGateway`转发音频到实时 ASR 管线
- ASR 支持三种运行形态：
 - `inprocess`：ASR 管线运行在 Web 主进程内
 - `process`：Web 主进程自动拉起本地 ASR 子进程
 - `remote`：连接外部独立 ASR worker，支持多 worker 固定路由
- 支持录音前确认提示、录音中麦克风热切换
- 同一课程名 15 分钟内再次开始录音时，前端可提示是否接续上一段课堂结果
- 转写结果写入：
 - PostgreSQL / SQLite`transcript\_records`（由`DATABASE\_BACKEND`决定）
 - 本地 JSONL 转写文件
 - Qdrant 向量库（实时或准实时）
- 前端可围绕当前课堂做 RAG 查询

1.2 课堂视觉能力

- 支持在视频画面上框选区域
- `ppt` 区域走 PaddleOCR
- `blackboard` 区域走 Qwen2.5-VL
- 视觉结果会写入统一 transcript 体系，再进入 RAG

1.3 视频处理与回放

- 前端可录制课堂视频并上传
- 后端将视频抽音频并调用 FunASR 生成字幕
- 字幕段落写入：
 - `lesson\_videos`
 - `transcript\_records`
 - Qdrant
- 历史页支持视频回放、字幕展示、字幕跳转

1.4 课件与资料解析

- 支持上传 PDF、PPT、Word、HTML、图片等课堂资料
- 支持两类入口：
 - 跟随某个课堂 session 上传的资料
 - 独立于课堂的资料库上传入口
- 后端通过 MinerU 解析文本、页面结构、公式和表格
- 解析结果写入统一 transcript / RAG 体系

- 问答时可由前端显式选择资料库文件作为辅助检索范围

1.5 课后复习能力

- 历史课堂浏览
- 原始转写与精修转写
- 历史课堂追问
- 课后总结
- 测验题生成
- 课节级笔记 `lesson\_notes`
- Markdown 导出

1.6 Lesson Copilot

当前已经有一个最小可用的 lesson copilot，入口在 `/workshop`。

它是一个窄域 agent，不负责整个系统调度，只围绕一节课做学习编排。当前可调用的工具包括：

- `get\_lesson\_note`
- `generate\_lesson\_note`
- `get\_lesson\_transcripts`
- `get\_refined\_lesson\_transcripts`
- `get\_lesson\_videos`
- `get\_session\_assets`
- `get\_lesson\_messages`
- `generate\_lesson\_quiz`
- `generate\_lesson\_summary`
- `query\_lesson\_knowledge`

1.7 Session 与运行时状态

- 全系统围绕 `course\_id + lesson\_id + session\_id` 组织课堂数据
- `session\_id` 用于单次录音 / 单次视频 / 单次实时链路
- `lesson\_id` 用于把同一节课的多段录音、多份视频、多轮问答串起来
- Session backend 支持两种模式：
 - 默认内存模式
 - 配置 `REDIS\_URL` 后切换到 Redis
- Redis 模式下会话状态带 TTL：
 - 活跃态默认保留 24 小时
 - 停止 / 错误态默认保留 6 小时

2. 系统页面

前端路由：

- `/`：实时课堂页 `LiveView`
- `/history`：历史回顾页 `History`
- `/workshop`：学习资料库 / 课后工作台 `StudyLibView`

推荐理解方式：

- `LiveView`：上课中
- `History`：回看和追溯
- `StudyLibView`：资料库、课后复习、笔记、copilot

3. 核心业务流

3.1 实时语音转写流

```
```text
浏览器麦克风
-> WebSocket /ws/audio/{session_id}
```

```
-> realtime_speech_service
-> AsrGateway
-> in-process ASR / local worker / remote worker
-> partial_transcript / final_transcript
-> transcript_records + JSONL
-> realtime_rag_indexer
-> 前端实时展示 / 课堂问答
...
```

### ### 3.2 视频字幕流

```
```text
浏览器录制视频
-> POST /sessions/{session_id}/videos
-> session_video_service
-> ffmpeg 抽音频
-> FunASR 离线转写
-> lesson_videos + transcript_records
-> rebuild_session_index
-> 历史页回放
...
```
```

### ### 3.3 视觉 OCR/VLM 流

```
```text
视频帧截图
-> POST /sessions/{session_id}/vision-frame
-> session_vision_service
-> OCR / VLM
-> transcript_records
-> RAG
...
```
```

### ### 3.4 课件资料流

```
```text
上传 PDF/PPT/HTML/图片
-> POST /sessions/{session_id}/assets
-> lesson_asset_service
-> MinerU 解析
-> transcript_records
-> RAG
...
```
```

资料库模式:

```
```text
上传 PDF/PPT/HTML/图片
-> POST /sessions/assets
-> lesson_asset_service
-> MinerU 解析
-> transcript_records
-> 在问答时按 asset_ids 参与辅助检索
...
```
```

### ### 3.5 课后笔记流

```
```text
POST /lessons/{course_id}/{lesson_id}/notes/generate
-> LessonNoteService
-> 聚合 lesson transcript_records
-> LLM 分块生成结构化笔记
```
```

```
-> lesson_notes 持久化
-> StudyLibView 展示 / 导出 Markdown
...
```

### ### 3.6 Copilot 流

```
```text
用户消息
-> POST /lessons/{course_id}/{lesson_id}/copilot
-> LessonCopilotService
-> LLM 决策调用工具
-> 工具结果回填
-> 最终回答 + steps
```
```

## ## 4. 技术栈

### ### 后端

- Python 3.10+
- FastAPI
- WebSocket
- PostgreSQL / SQLite
- Redis (可选, 用于 session backend)
- Qdrant
- LlamaIndex
- OpenAI-compatible LLM API

### ### 模型与 AI 能力

- 实时 / 离线 ASR: FunASR / Paraformer
- VAD: FSMN VAD / Silero VAD
- 标点: CT-Transformer
- Embedding: `BAAI/bge-small-zh-v1.5`
- OCR: PaddleOCR
- VLM: `Qwen/Qwen2.5-VL-7B-Instruct`
- 目标检测: YOLO11
- 文档解析: MinerU

### ### 前端

- Vue 3
- TypeScript
- Vite
- Pinia
- Vue Router
- Tailwind CSS

## ## 5. 项目结构

```
```text
.
├─ config/
│  ├─ settings.py      # 全局配置
│  └─ prompts.py       # 笔记 / quiz / summary / RAG / copilot prompt
├─ data/               # 运行时数据
├─ models/             # 本地模型目录
├─ scripts/            # 模型下载、RAG 脚本、copilot 运行脚本
├─ src/
│  ├─ application/     # 业务用例层
│  │  ├─ chat/
│  │  └─ documents/
```

```
| | └ live_classroom/
| | └ lesson_copilot/
| | └ lesson_notes/
| | └ rag/
| | └ review/
| | └ runtime/
| | └ speech/
| | └ transcripts/
| | └ video/
| └ core/          # 模型 / 音频 / 知识库核心能力
| └ domain/        # 领域对象
| └ infrastructure/ # 存储、日志、模型加载
└ tests/
  └ web/
    └ backend/
      └ app/
        └ api/      # FastAPI 路由, 按
sessions/history/assets/videos/learning 拆分
      └ dependencies.py # Web 侧服务装配与生命周期
      └ execution.py    # 同步重任务线程化执行
      └ presenters.py   # Web 响应转换
      └ schemas.py      # Web 请求模型
      └ uploads.py      # 上传流写入工具
      └ tasks.py        # Web 后台任务适配
    └ frontend/
      └ src/
...

```

6. 数据存储

6.1 关系数据库

业务数据库通过 `DATABASE\_BACKEND` 选择:

- `DATABASE\_BACKEND=postgresql`: 使用 PostgreSQL, 当前 `config/.env` 示例指向 `postgresql://study\_agent@127.0.0.1:5432/study\_agent`
- `DATABASE\_BACKEND=sqlite`: 使用本地 SQLite, 默认路径为 `data/study\_agent.sqlite3`

当前关键表:

- `chat\_messages`
- `transcript\_records`
- `lesson\_assets`
- `lesson\_videos`
- `refined\_transcript\_records`
- `lesson\_notes`

6.2 Redis (可选)

- 用途: session backend
- 开启方式: 配置 `REDIS\_URL`
- 关键 key:
 - `study-agent:sessions:index`
 - `study-agent:sessions:session:{session\_id}`
- 过期策略:
 - 活跃态默认 `86400` 秒
 - 停止 / 错误态默认 `21600` 秒

6.3 本地文件目录

```
```text
data/transcripts/ # JSONL 转写

```

```
data/assets/ # 原始资料文件
data/videos/ # 上传视频
data/video_subtitles/ # wav / srt
data/mineru_results/ # MinerU 解析结果
data/qdrant/ # 本地 Qdrant 数据
logs/ # 日志
...`
```

### ### 6.4 向量库

- 默认使用 Qdrant
- 支持本地 Qdrant 目录
- 主要索引内容来自统一 transcript 记录，而不是直接索引原始二进制文件
- 问答返回保留扁平 `results`，同时增加按来源归类的 `grouped\_results`
- 当前统一来源类型包括：
  - `speech`
  - `ocr`
  - `vlm`
  - `documents`
  - `other`

## ## 7. 环境要求

基础要求：

- Python 3.10+
- Node.js `^20.19.0 || >=22.12.0`
- npm
- Git

建议机器配置：

- 16GB+ 内存
- Windows 10/11、macOS 或 Linux
- 有独显更好，但不是必须

额外说明：

- 视频转字幕依赖 FFmpeg
- 首次运行会加载本地模型，冷启动较慢是正常现象
- OCR / VLM / Embedding 模型占用较大，Windows 机器建议保证足够页面文件

## ## 8. 安装与初始化

以下命令默认在仓库根目录执行。

### ### 8.1 创建虚拟环境

```
```powershell
python -m venv .venv
.\venv\Scripts\Activate.ps1
python -m pip install -U pip
```
```

如果 PowerShell 阻止脚本执行：

```
```powershell
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\venv\Scripts\Activate.ps1
```
```

### ### 8.2 安装 Python 依赖

```
```powershell
python -m pip install -r requirements-rag.txt
python -m pip install huggingface_hub modelscope pytest
```
```

### ### 8.3 下载本地模型

Windows:

```
```powershell
.\setup_model.bat
```
```

或直接运行:

```
```powershell
python scripts/setup_models.py
```
```

脚本会准备这些模型:

- Embedding: `models/embedding/bge-small-zh-v1.5`
- ASR: `speech\_paraformer-large-vad-punc\_asr\_nat-zh-cn-16k-common-vocab8404-pytorch`
- VAD: `speech\_fsmn\_vad\_zh-cn-16k-common-pytorch`
- Punctuation: `punc\_ct-transformer\_cn-en-common-vocab471067-large`
- YOLO: `yolo11s.pt`
- PaddleOCR det / rec
- Qwen2.5-VL-7B-Instruct

注意:

- 前端里也有 streaming 模型选项, 若你要用在线流式 Paraformer, 需要额外准备:

```
`models/asr/speech_paraformer-large_asr_nat-zh-cn-16k-common-vocab8404-online`
```

### ## 9. 配置

项目默认读取:

```
```text
config/.env
```
```

建议最少配置以下变量:

```
```env
# Database
DATABASE_BACKEND=postgresql
DATABASE_URL=postgresql://study_agent@127.0.0.1:5432/study_agent
DATABASE_POOL_MIN_SIZE=1
DATABASE_POOL_MAX_SIZE=20
DATABASE_POOL_TIMEOUT_SECONDS=10

# Session backend (可选)
REDIS_URL=
SESSION_REDIS_PREFIX=study-agent:sessions
SESSION_REDIS_TTL_SECONDS=86400
SESSION_REDIS_TERMINAL_TTL_SECONDS=21600

# Qdrant / RAG
```

```
RAG_QDRANT_PREFER_LOCAL=true
RAG_QDRANT_COLLECTION=speech_transcript_chunks
RAG_REALTIME_INDEXING_ENABLED=true
RAG_REALTIME_FLUSH_RECORDS=3
RAG_REALTIME_FLUSH_CHARS=300
RAG_REALTIME_FLUSH_INTERVAL_SECONDS=20
```

```
# Realtime ASR worker
ASR_RUNTIME_BACKEND=process
ASR_WORKER_QUEUE_SIZE=512
ASR_WORKER_CLOSE_TIMEOUT_SECONDS=5
ASR_WORKER_CONNECT_TIMEOUT_SECONDS=5
ASR_WORKER_HOST=127.0.0.1
ASR_WORKER_PORT=8765
ASR_WORKER_ENDPOINTS=
ASR_WORKER_AUTH_TOKEN=study-agent-asr
```

```
# LLM (建议用于笔记、测验、summary、copilot)
RAG_ENABLE_LLM=true
RAG_LLM_PROVIDER=deepseek
RAG_LLM_MODEL=deepseek-chat
RAG_LLM_API_KEY=your_api_key
RAG_LLM_API_BASE=https://api.deepseek.com
RAG_LLM_TEMPERATURE=0.1
RAG_LLM_MAX_TOKENS=512
```

```
# MinerU (资料解析)
MINERU_API_TOKEN=your_mineru_token
MINERU_MODEL_VERSION=vlm
MINERU_LANGUAGE=ch
MINERU_ENABLE_FORMULA=true
MINERU_ENABLE_TABLE=true
MINERU_IS_OCR=false
MINERU_AUTO_INDEX_ENABLED=true
...
```

补充说明:

- `ASR\_DEVICE=auto`、`EMBED\_DEVICE=auto` 会自动解析为可用的 `cuda` 或 `cpu`
- `DATABASE\_BACKEND=postgresql` 时, 所有业务表写入 PostgreSQL; `DATABASE\_BACKEND=sqlite` 时写入本地 SQLite
- 实时 ASR 的数据库写入发生在 Web 主服务侧, ASR worker 只负责音频处理和返回转写事件
- `ASR\_RUNTIME\_BACKEND=process` 会由 Web 主服务自动拉起本地 ASR 子进程
- `ASR\_RUNTIME\_BACKEND=remote` 会连接独立 ASR worker; 配置 `ASR\_WORKER\_ENDPOINTS` 后会启用多 worker, 并按 `session\_id` 固定路由
- 不需要 LLM 时, 可设 `RAG\_ENABLE\_LLM=false`
- 不配置 MinerU 时, 实时课堂和普通 RAG 仍可运行, 但资料解析功能不可用
- 配置 `REDIS\_URL` 但 Redis 未启动时, 后端会在 startup 阶段直接失败
- `config/settings.py` 是当前所有默认路径和运行参数的最终来源

10. 启动方式

10.1 启动后端

```
```powershell
python -m uvicorn web.backend.main:app --host 127.0.0.1 --port 8000 --reload
```
```

独立 ASR worker 模式:

```
```powershell
python scripts/run_asr_worker.py --host 127.0.0.1 --port 8765
```



...

并在 `config/.env` 中配置：

```
```env
ASR_RUNTIME_BACKEND=remote
ASR_WORKER_HOST=127.0.0.1
ASR_WORKER_PORT=8765
ASR_WORKER_AUTH_TOKEN=study-agent-asr
```
```

多个独立 ASR worker：

```
```env
ASR_RUNTIME_BACKEND=remote
ASR_WORKER_ENDPOINTS=127.0.0.1:8765,127.0.0.1:8766
ASR_WORKER_AUTH_TOKEN=study-agent-asr
```
```

健康检查：

```
```text
GET http://127.0.0.1:8000/
```
```

正常返回：

```
```json
{"status":"ok"}
```
```

ASR worker 状态：

```
```text
GET http://127.0.0.1:8000/asr/status
```
```

示例返回：

```
```json
{"backend":"process","healthy":true,"worker_pid":12345,"active_session_count":0}
```
```

### 10.2 启动前端

```
```powershell
cd web\frontend
npm install
npm run dev
```
```

默认前端地址通常是：

```
```text
http://127.0.0.1:5173
```
```

如果端口被占用，Vite 会自动切到别的端口。

如需指定后端地址，可在 `web/frontend/.env.local` 中配置：

```
```env
VITE_API_BASE_URL=http://127.0.0.1:8000
```
```

## ## 11. 使用方式

### ### 11.1 实时课堂

1. 打开 `/`
2. 创建或进入课堂
3. 允许浏览器麦克风权限
4. 开始录音
5. 查看实时转写
6. 在课堂问答区提问

补充：

- 如果同名课程 15 分钟内存在上一段录音快照，前端会提示是否接续
- 接续时会沿用 `course\_id + lesson\_id`，但会创建新的 `session\_id`
- 课堂问答默认是“课堂上下文直答”模式；打开 `include\_rag\_context` 后会返回结构化检索结果与引用

### ### 11.2 上传资料

1. 在课堂中上传 PDF / PPT / Word / HTML / 图片
2. 等待 MinerU 解析完成
3. 再在问答区或历史页使用这些内容

如果是资料库模式：

1. 打开 `/workshop`
2. 上传资料到共享资料库
3. 在课堂问答区勾选需要参与检索的资料
4. 再发起课堂问答

### ### 11.3 上传视频

1. 在课堂中录制视频
2. 停止录制后上传
3. 后端离线抽字幕
4. 到历史页回放

### ### 11.4 课后复习

1. 打开 `/history`
2. 查看历史课节
3. 看原始转写 / 精修转写 / 视频

### ### 11.5 笔记与 Copilot

1. 打开 `/workshop`
2. 左侧选择课节
3. 生成课后笔记或重新生成
4. 导出 Markdown
5. 在 `Lesson Copilot` 面板里直接提复习请求

## ## 12. 主要 API

### ### 12.1 WebSocket

```
```text
ws://127.0.0.1:8000/ws/audio/{session_id}
```
```

实时 ASR 状态:

```
```text
GET /asr/status
```
```

### ### 12.2 Session / History

```
```text
POST /sessions
GET /sessions
GET /sessions/history
GET /sessions/history/messages
GET /sessions/history/transcripts
GET /sessions/history/refined-transcripts
GET /sessions/history/videos
GET /sessions/{session_id}/transcripts
GET /sessions/{session_id}/messages
```
```

### ### 12.3 资料 / 视频 / 视觉

```
```text
GET /sessions/assets
POST /sessions/assets

POST /sessions/{session_id}/assets
GET /sessions/{session_id}/assets
GET /sessions/assets/{asset_id}

POST /sessions/{session_id}/videos
GET /sessions/{session_id}/videos
GET /sessions/videos/{video_id}
GET /sessions/videos/{video_id}/file
GET /sessions/videos/{video_id}/srt

POST /sessions/{session_id}/vision-frame
```
```

### ### 12.4 问答 / 总结 / 测验

```
```text
POST /sessions/{session_id}/query
POST /sessions/{session_id}/summary
POST /sessions/{session_id}/quiz
```
```

`POST /sessions/{session\_id}/query` 当前支持的重要参数:

- `scope`: `auto | current\_lesson | course\_all | course\_history | global`
- `with\_llm`
- `include\_rag\_context`
- `classroom\_context\_mode`: `session | lesson`
- `asset\_ids`: 显式限制资料库检索范围

返回中除了 `answer / results / citations / metadata`, 还会带:

- `grouped\_results`
- `result.source\_kind`
- `citation.source\_kind`

## ### 12.5 Lesson Notes / Copilot

```
```text
GET /lessons/notes/{note_id}
GET /lessons/{course_id}/{lesson_id}/notes/latest
POST /lessons/{course_id}/{lesson_id}/notes/generate
POST /lessons/{course_id}/{lesson_id}/copilot
```
```

## ## 13. 脚本

```
```text
scripts/setup_models.py      # 下载本地模型
scripts/rag_build_index.py   # 构建 RAG 索引
scripts/rag_query.py         # 命令行查询 RAG
scripts/rag_eval.py          # 评测 RAG
scripts/video_to_srt.py      # 视频转字幕
scripts/run_lesson_copilot.py # 命令行运行 lesson copilot
scripts/run_asr_worker.py    # 独立实时 ASR worker
```
```

## ## 14. 测试与检查

## ### 后端测试

```
```powershell
python -m pytest -q
```
```

## ### 前端类型检查

```
```powershell
cd web\frontend
npm run type-check
```
```

## ### 前端构建

```
```powershell
cd web\frontend
npm run build
```
```

## 推荐检查顺序：

1. `python -m py\_compile config/settings.py web/backend/main.py`
2. `python -m pytest -q`
3. `cd web/frontend && npm run type-check`
4. 启动前后端手工走一遍：
  - 实时录音
  - 上传课件
  - 上传视频
  - 生成笔记
  - Ask Copilot

## ## 15. 常见问题

## ### 15.1 后端启动很慢

## 原因通常是：

- 使用 `ASR\_RUNTIME\_BACKEND=inprocess` 时，Web 主进程首次加载 ASR / FunASR

- 使用 `ASR\_RUNTIME\_BACKEND=process` 或 `remote` 时, ASR 会在本地子进程、独立 worker 或首次录音时加载
- 首次加载 embedding / OCR / VLM
- 本地磁盘或内存不足

建议演示前提前启动一次, 并访问 `/asr/status` 确认 worker 可用。

### ### 15.2 LLM 调用失败

检查:

- `RAG\_ENABLE\_LLM=true`
- `RAG\_LLM\_API\_KEY` 是否配置
- `RAG\_LLM\_API\_BASE` 是否正确
- 供应商余额是否足够

### ### 15.3 Windows 出现 SciPy / transformers / 页面文件错误

如果出现类似:

```
```text
DLL load failed
页面文件太小, 无法完成操作
```
```

通常是:

- 页面文件太小
- 内存压力过高
- 一次性导入了过重依赖

建议增大页面文件, 并避免同时启动过多模型任务。

### ### 15.4 资料上传后无法检索

检查:

- `MINERU\_API\_TOKEN` 是否有效
- `MINERU\_AUTO\_INDEX\_ENABLED=true`
- 资料状态是否已到 `done`
- `lesson\_assets.record\_count` 是否大于 0

### ### 15.5 视频有回放但没有字幕

检查:

- FFmpeg 是否可用
- FunASR 模型是否完整
- `lesson\_videos.status` 是否为 `done`

### ### 15.6 启动时报 Redis 连接拒绝

如果出现:

```
```text
ConnectionError: Error 10061 connecting to 127.0.0.1:6379
```
```

通常说明:

- `config/.env` 中配置了 `REDIS\_URL`
- 但本机 Redis 或 Docker 容器未启动

处理方式:

- 启动 Redis
- 或临时移除 `REDIS\_URL`, 退回内存 session backend

### 15.7 SentenceTransformer / torch 不支持 auto

当前 embedding 设备选择已经做了自动解析:

- `EMBED\_DEVICE=auto` 会被解析为 `cuda` 或 `cpu`
- 如果仍然报 device 相关错误, 优先检查:
  - 依赖是否完整
  - CUDA 是否可用
  - 本地模型目录是否存在

## 16. 当前定位

从架构上看, 这个项目当前更接近:

```
```text
实时课堂系统 + 课后复习平台 + lesson copilot
```
```

而不是单纯的通用 agent 框架。

建议继续沿着这三条业务线演进:

1. 实时课堂
2. 课后复习
3. 学习工作台 / copilot

agent 只作为上层编排层, 不要反过来成为整个系统骨架。

## 17. 安全与数据注意事项

- 不要把真实 `.env`、API Key、数据库、日志和模型文件提交到仓库
- MinerU 会把上传文件发往外部服务, 敏感资料先脱敏
- 公开部署时应补充鉴权、HTTPS、限流和存储隔离
- 演示用课堂数据建议与真实教学数据分开

## 18. 许可证与说明

本仓库中的第三方模型、数据和 API 依赖各自遵循它们自己的许可证和使用条款。正式部署前, 请分别核对:

- FunASR / Paraformer
- BGE
- PaddleOCR
- Qwen2.5-VL
- YOLO11
- MinerU
- DeepSeek / OpenAI-compatible API